Enhancing Playlist Recommendations with Similarity-Weighted Scoring and Diversity

Ziheng (Tony) Fang
CS 274
San Jose State University
San Jose, USA
tony.fang@sjsu.edu

***Abstract*—Whether it's Apple Music or Spotify, recommendation systems are core components of most music streaming platforms in 2026. In this work, we extend a baseline recommendation system that uses the semantic meanings of playlist titles to recommend songs from other playlists that are "similar" in playlist title semantics. Upon working with the baseline system, we found two limitations where all similar playlists were weighted "equal" and the lack of diverse recommendations. We addressed these two issues by introducing weighted scoring for similar playlists where playlists that were more similar had more influence as well as limiting the number of recommendations that could be made per individual artist, ensuring that the user gets a diverse set of recommendations.**

Keywords—recommendation, music, Spotify, playlists, similarity

I. Introduction

Across all music streaming platforms in 2026, a recommendation system is a crucial component. Traditional approaches such as content-based recommendations and collaborative filtering do not capture semantic meaning of user generated playlist titles. Advances in large language models (LLMs) have allowed us to address that issue. When it comes to recommending music, the title of a playlist provides very valuable information as they often indicate the mood of the user or what the user intends for the playlist. By analyzing playlist titles by semantic meaning, it makes it possible to identify playlists created for similar contexts.

We found a baseline system, the LLM Playlist Recommender which utilizes an LLM to identify similar playlists by title. Songs are then recommended based on their frequency of occurrence across similar playlists. Upon analyzing the baseline system, we found two limitations that could be addressed in a school semester, the strategy of all similar playlists being treated as equal and the lack of diversity of songs.

By using the k-means algorithm to find similar playlists, we an always find k playlists that are "similar" to the current playlist. This does not take into account that we may have a few playlists that are truly similar to the playlist we just created, and the algorithm would include other playlists that do not share much similarity so we would have k-similar playlists. Allowing equal weight across all these playlists would allow potentially unrelated playlists to contribute to recommendations as much as playlists that are truly similar. With weighting, we ensure that only playlists that are truly similar have a say in which songs are recommended.

Across all streaming platforms, it is also very evident that a few popular artists dominate the platform. It is then likely that some artists simply appear more often in other playlists simply due to popularity, not similarity of what the user is looking for. This could potentially limit the content user discovers. By limiting the number of songs that can be recommended from each artist, we ensure that the user is not limited to hearing songs from a select group of popular artists.

This work is closely aligned to recommendation systems in general within Web Intelligence. Recommendation systems enable content providers to deliver personalized content towards users, which in our case is music. The latest developments in the area includes the ability to analyze semantic relationships in user-generated content. In this work, an LLM is utilized to analyze semantic meaning behind playlist titles [3].

In subsequent sections, we dive into the data utilized to train our model, the architecture of the baseline system, improvements we introduced to the baseline system, as well as possible improvements that could be done in future work.

II. Dataset & Preprocessing

A. Spotify MPD

The MPD [1] refers to the Million Playlist Dataset from Spotify used specifically for AI research. The MPD was in JSON format and contains 1M user generated Spotify playlists, originally used for the RecSys challenge. Each playlist contains a title, included tracks, as well as metadata, making the dataset a perfect for building a recommendation system.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

The MPD is originally in JSON format, where a subset of playlists are included in each file. Individual playlist entries include a playlist identifier, playlist title, along with a list of tracks in the playlist. Each track is also associated with metadata such as the track name, artist name, and album information.

Specific to playlists that are created by users, titles of these playlists often reflect mood, theme, or context as the playlist titles are user defined. This makes the dataset particularly suitable for semantic-based similarity among playlist titles.

The scale and diversity of the MPD gives both opportunities and challenges for the development of a recommender system. The large number of playlists from a vast amount of users would allow a model to detect patterns. However, we also could not rule out the presence of noise due to differences in how users name titles causing ambiguity.

B. *JSON to CSV*

In order to enable efficient computation of the data, we transformed the dataset into tabular format. The process was done by the conversion of original JSON files into multiple CSV files, which included:

- playlists.csv: Playlist-level metadata.
- items.csv: Map playlists to their tracks.
- tracks.csv: Stores metadata for individual tracks.

The transformation of the dataset into tabular format allows faster access and simplified processing during later stages of the pipeline. In particular, it allows for faster lookup of playlist titles and metadata.

During the implementation, the preprocessing stage involved careful handling of file paths when parsing large volumes of JSON data.

III. Baseline System Architecture

The original baseline system follows multiple stages that involves transforming data from the MPD into semantic representations and generates recommendations for a new playlist based on similarity between other playlists and the aggregation of tracks in these playlists. The system utilizes the playlist titles to identify similar playlists.

A. *Playlist Embedding*

The first stage of the pipeline converts the titles of playlists into vectors using a language model, SentenceBERT [2]. The model produces embedding vectors that contain the semantic meaning of individual playlist titles. Semantically similar playlist titles would be close to each other in the vector space.

This vector representation allows identification of relationships between playlists when their titles contain completely different words. For example, two playlists titled “party vibes” and “fun night” may be very close together in the vector space due to similar semantic meaning, even though the titles contain very different words. On the other hand, “work mix” and “workout mix” have very similar wording, but semantic embedding would put these titles far apart.

B. *Similar Playlist Retrieval*

When we have a query playlist, or a playlist that we just created, the system would compute the embedding vector of its own title. The embedding vector would then be compared to the embedding vectors of all other embedding vectors in the dataset. Cosine similarity is used to measure similarity among the vectors [4].

For each playlist in the dataset, we compute a cosine similarity between it and the query playlist. The playlists across the MPD are then ranked in similarity, we then utilize the “top-k” similar playlists. These playlists are then used for generating recommendations.

C. *Song Aggregation and Ranking*

After obtaining the top-k most similar playlists, the system would aggregate tracks that appear within these playlists. Under the baseline system, each of these playlists contributes its tracks equally. Appearance frequency of each track is used to determine the importance of individual tracks.

Formally, the score of a track is expressed as:

$$S = \sum 1$$

S is the score of each individual track. The score is incremented by 1 each time a song appears in any of the top-k playlists. The top-N songs are then used as final recommendations.

This aggregation process can also be viewed as a voting mechanism. Each playlist has an equal contribution to the ranking of songs. A track that appear in more similar playlists would end up with a higher score, which would increase the likelihood of it being recommended. For example, if we identify a song that appears in 5 of the k similar playlists, that song would get a score of 5. Whereas, if another song were to appear in 3 of the k similar playlists, it would only get a score of 3.

D. Recommendation Output

The final stage of the pipeline involves recommending tracks to the user. Individual recommendation includes track titles along with artist names. Occurrence counts across similar playlists are also included. This process allows a user to see why a particular track was recommended to them.

IV. Improvements From Baseline

This baseline system is able to utilize the semantics of playlist titles to identify similar playlists and recommend popular songs from these similar playlists. However, upon examination, there were many limitations of this baseline system. Two improvements that were feasible for a semester long project included addressing weighing similar playlists by how similar and obtaining more diverse recommendations. We implemented a scoring algorithm where similar playlists were scored based on the similarity and ranking similar songs with a limitation on how many songs from a single artist can be recommended.

A. Weighing "Similar" Playlists

The baseline system finds k-similar playlists and treats all the k playlists equally. Each track is assigned a score based on the frequency of appearances among the k similar playlists. Specifically:

$$\text{Score} = \sum 1$$

Should all similar playlists be equally similar, this approach would be perfect. However, in real life situations, this is rarely the case. When we look for k playlists most similar to our query playlist, our k-means algorithm would return k playlists with the highest cosine similarity. A few playlist may be significantly more similar to our query playlist than the rest. In extreme situations, there may be only one playlist truly "similar" to our query playlist. Our algorithm would be forced to return k playlists, where the rest may have the "highest" of low similarity scores. These playlists would have little relevance in reality. To address this issue, we implemented an algorithm that scores each playlist based on similarity. Rather than an equal point per appearance in any of the top-k similar playlists, each playlist contributes a "score" of its similarity with the query playlist. In particular:

$$\text{Score} = \sum \text{similarity(playlist)}$$

If we considered a scenario where we identified k similar playlists to our query playlist, one of which has a similarity score of 0.95, while all others have a score of <0.1. Under the baseline system, all k playlists would have an equal contribution to the recommendations. However, we can clearly see that there is only one playlist truly "similar" to our query playlist. Thus, the approach by the baseline system would very likely end up producing irrelevant recommendations. With our extension, we make sure that songs appearing in the playlist with the similarity score of 0.95 would dominate over the ones that have a similarity of <0.1.

B. Diversity-Aware Recommendations

The ability to discover new content on a platform makes the experience better for many users on any streaming platform [5]. We identified an issue in the baseline system where diversity was never taken into account in providing recommendations. Songs from more popular artists are expected to occur in playlists at a higher frequency [6]. Thus by recommending songs solely based on frequency, we unknowingly favor popular artists. The approach by the baseline system does not address this popularity bias.

In order to address this issue, the recommendation algorithm was modified when it comes to the artist. Upon obtaining a list of high-frequency songs, the system introduces a constraint on the number of songs that can be recommended from the same artist. The procedure is as follow:

1. Constraint M is introduced, which is the maximum number of tracks from each individual artist.
2. As in the baseline system, we retrieve the candidates of top-ranked songs.
3. The list is sorted by descending order based on the score.
4. We add songs to our playlist only if the artist has not exceeded the constraint of M.

```
Algorithm: Diversity-Aware Generator
Input(s): List of ranked songs, Constraint limiting number of songs per artist
Output: Top-N Recommendations

Initialize empty results
For each song in the ranked list:
        if artistCount < M:
                add song to results
        if result_list.size == N
                break
return result_list
```

With a constraint on artist frequency, the system addresses the artist popularity bias and forces recommendations to go towards another artist if one artist has already reached the constraint.

C. Combined Improvements

These improvements would enhance the user experience in unique ways. Similarity-weighted scoring of similar playlists would ensure that recommendations take more input from playlists that are more relevant. Incorporating diversity allows the user to discover new content that they would not have otherwise. Both improvements are also feasible for a semester project as they do not require massive data collection or training/fine-tuning new models.

V. Implementation Details

The improvements on diversity and relevance involved modifications at the recommendation stage of the original pipeline. Stages such as dataset preprocessing, clustering, embedding generation, etc., were not modified.

The baseline system first identifies the top-k playlists most similar to a query playlist based on title semantics. Each playlist was then stored as a tuple containing the playlist identifier and the cosine similarity score. The baseline implementation did not make use of the similarity score and allowed all similar playlists to contribute equally.

To implement similarity weighted scoring, we modified the aggregation loop so that each occurrence would contribute the similarity score of the playlist it appeared in rather than a uniform value.

```
-   for pid, 1 in similar_playlists: ## loop through each tuple in similar playlists
+   for pid, sim in similar_playlists: ## loop through each tuple in similar playlists
        pid_str = str(pid) # convert to string
        if pid_str in playlist_tracks: # check if the playlist id is present in playlist track dictionary
            for track_metadata in playlist_tracks[pid_str]: # loop through each track metadata in the playlist
-               song_counter[(track_metadata["track_name"], track_metadata["artist_name"])] += 1 # increment count of that song and artist
+               song_counter[(track_metadata["track_name"], track_metadata["artist_name"])] += sim # increment count of that song and artistby similarity
```

In particular, the baseline system had:

song_counter[(song, artist)] += 1

Our new implementation has:

song_counter[(song, artist)] += similarity

Separately, a modification was made to how songs are recommended from the list of candidates. We would still generate a list of highly ranked songs. However, we track how many times an artist has their song recommended. A song would only end up in the recommendations if the artist has not exceeded our constraint.

```
+ # 7) Add diversity constraint to songs recommended
+ ######################################
+
+ def get_diverse_top_songs_with_artists(similar_playlists, playlist_tracks, top_k=10, max_songs_per_artist=1, candidate_pool=50):
+     song_counter = Counter()
+
+     for pid, _ in similar_playlists: ## loop through each tuple in similar playlists
+         pid_str = str(pid) # convert to string
+         if pid_str in playlist_tracks: # check if the playlist id is present in playlist track dictionary
+             for track_metadata in playlist_tracks[pid_str]: # loop through each track metadata in the playlist
+                 song_counter[(track_metadata["track_name"], track_metadata["artist_name"])] += 1 # increment count of that song and artist
+
+     ranked_songs = song_counter.most_common(candidate_pool) # most common songs
+
+     diverse_results = []
+     artist_counts = Counter() # counts of how many times each artist has been included
+
+     for (song, artist), count in ranked_songs: # loop through all the top songs and artists
+         if artist_counts[artist] < max_songs_per_artist: # if the artist has not yet had the max songs
+             diverse_results.append(((song, artist), count)) # add the song to the diverse results
+             artist_counts[artist] += 1 #increment the count of songs for that artist
+
+         if len(diverse_results) >= top_k: # if we already have the number of wanted songs
+             break
+
+     return diverse_results
```

VI. Results/Observations

The evaluation of our extension was done through the comparison of recommendations provided by the baseline system and the implementation of our extension. We had a few individuals "test" sample query playlist names and review songs that were then recommended to the test playlist. We then took note of feedback from these individuals.

A. Diversity Impact

We tested the baseline system on a few "test" query playlists and observed that the recommendations it produced were dominated by a small set of popular artists. This behavior is expected as popular artists are more widely known and are more likely to appear in any playlist, including the top-k playlists.

Our approach allows the user to specifically limit the number of songs that can be recommended from an individual artist. This forces the recommendation system to look into other artists once certain artists already cross the constraint threshold. At the end, recommendations from our approach would contain a more wide variety of artists.

Whereas the baseline system often included multiple songs from the same artist, our diversity aware approach often included a wide variety of artists in our recommendations. This allows the user to more easily discover new content.

B. Relevance by Similarity

In the baseline system, once the k-means algorithm identified the top-k similar playlists, each of the k playlists would contribute equally when it comes to which songs were recommended to the query playlist. This assumes that the top k playlists are equally similar, which is often not the case.

Our similarity-weighted approach takes into account that some of the k playlists can be significantly more similar to our query playlist than others. In certain situations, while the k-means algorithm will find the top-k similar playlists, only a handful of the k are truly similar. Our approach was to give songs that appear in more similar playlists a higher score than songs appearing in less similar playlists. At the end, we would have recommendations influenced more by the most similar playlists, based on title semantics.

However, after running multiple tests, we were not able to find too much of a difference between the extension and the baseline system. We suspect that this is due to the fact that our "sample" query playlists did not have a situation that of the k most similar playlists, some of the k playlists were more semantically similar than others. The improvement could be replicated in the future by creating a playlist name that has a significant difference in similarity among the k most similar playlists. This finding also suggests that the effectiveness of our similarity-weighted scoring algorithm relies heavily on how similarity scores vary among the top k most similar playlists.

C. Combined Improvements

At the end, recommendations created by our extension were observed to be more varied. This allows users to more easily discover new music. By utilizing a weighing on similar playlists based on the similarity,

we also ensure that recommendations are more relevant to the user.

Our approach resulted in recommendations that were more relevant and diverse.

D. Potential Tradeoffs

The extension produced significant improvements when it came to the diversity of the recommendations and relevance, we do believe that they also introduced certain limitations as well. In particular, when a diversity constraint is implemented, we could very well be excluding a song that is more relevant to what a user is looking for than a song the user actually gets from an artist that hasn't appeared as often.

This points to guidance that diversity constraints should be tuned so that users do get a diverse set of recommendations but are not unintentionally leaving out highly relevant songs from an artist that has already appeared in the playlist.

LIMITATIONS & FUTURE WORK

Although the extension gave significant improvement in recommendations from relevance and diversity, several additional limitations remain. Possible improvements that future work can address on include personality aware features, further semantic analysis, as well as having the model learn what to do instead of rigid rules through heuristics.

Something that we noticed was that the recommendation system relies on identifying playlists that are similar in title semantics and recommending songs that appear in those playlists. A problem could arise when a user creates a playlist titled with something that would be unique to their understanding, introducing ambiguity. For example, a playlist titled "workout music" would allow our system to easily find playlists for the same motive. However, a playlist titled "My Favorite Songs", "Playlist for the Special Day", etc. would make looking into similar playlists irrelevant. Since we already generate semantic understanding of playlist titles using SentenceBERT, something that could be added to the pipeline is prior to k-means finding similar playlists, we could use SentenceBERT to evaluate whether the playlist is user-specific. If so, rather than looking for "similar" playlists from *other* users, we could recommend songs based on the current user's own musical taste. Such a limitation would reduce the accuracy of identifying similar playlists, as many similar playlists picked up could be irrelevant, which leaves recommendation quality highly dependent on the titles provided.

Another key limitation was something that was identified at the beginning of the semester, which was that the recommendation system completely ignores individual personalities when making recommendations. We rely solely on playlist titles. No user specific information such as interests, listening history, or demographics are taken into account. This could end up significantly impacting the relevance of recommendations that are later generated. For example, if we look at listening history, someone who listens to songs from High School Musical frequently would probably find songs from the series more relevant when they look for "party music" as compared to someone who enjoys Hannah Montanah. Furthermore, by looking into the individual interests of users, one can see that someone who is into K-Pop would find BTS music more appropriate for "Summer Break Vibes" as compared to someone who doesn't know anything about K-Pop. As we all know, different ages prefer very different types of music. A senior would find "relaxing music" very different than something a young adult or teenager would find.

To address personality-aware recommendations, our model could be modified where instead of relying solely on playlist title semantics, we would also factor in user similarity. User details could be vectorized (e.g. interests, demographics, etc.). Similarity between users that created the playlist and the playlist titles themselves could both be taken into account when choosing the top-k similar playlists.

For a semester long project, a bottleneck that prevented implementation of a personality aware feature was the need to obtain large scale data based on preferences of users and utilizing that data to train and test a model that was based off of individual personalities. Future work could entail gathering data from direct surveys towards a large set of diverse users where user demographic (e.g. age, gender, ethnicity, etc.) is recorded along with songs they enjoy. Another approach could be to enable the user to connect their social media accounts to the recommender system, which would enable the recommender system to get demographic detail in addition to personal interests.

Furthermore, our improvements on the system still rely on heuristic-based methods, which in our case are pre-defined rules based on similarity of playlists and frequency within those similar playlists. This approach does not allow our model to learn from data and would miss more relationships that we did not have in mind during the design. With rigid rules, the model is not able to adapt to dynamic situations. For example, our current implementation introduced a strict constraint where the number of songs for a single artist cannot exceed a threshold. However, in certain context, such as if a user was creating a playlist for an event related to a particular artist, our recommendation system would clearly not be suitable as the user would only get a limited number of songs from the artist that they are looking for. A system with a model that learns the context would be able to recommend diverse songs when a user is exploring new content and songs from a particular artists for playlists pertaining to that artist.

Finally, our implementation faced hardware constraints related to computational resources. Stages such as the model fine-tuning were simplified. Future work could involve training and fine tuning on a setup that has a compatible GPU present.

CONCLUSION

Recommendation systems are a core component of Web Intelligence [7]. Across different industries, whether its enabling personalized content on a user's news feed, finding related videos, finding products a user might enjoy in e-commerce, or new streaming content, recommendation systems play a crucial role. Recent developments in the field include the ability for recommendation systems to capture semantic relationships between content that users create. The recommendation system extended upon by this work closely relates with these techniques as a language model was utilized to create embedding vectors, which were then utilized to calculate similarity among individual playlists.

This work involved the improvement of a baseline recommendation system that recommended songs to a user when they created a new playlist. The new playlist was used as a "query" playlist where similar songs were identified through other playlists that had semantically similar playlist titles.

Upon successful implementation of the baseline system, we introduced two extensions which included more diverse recommendations and higher relevant songs. By introducing a constraint to limit the number of songs from each individual artist that can be recommended, the recommendation system would be less likely to generate recommendations from a single artist or a few artists. Thus, the user would be more likely to find new content. Instead of scoring each song by the appearance frequency among all top-k similar playlists, a song gets a higher score for appearing in a similar playlist that has a higher similarity score. This addresses the issue where only a few of the top-k playlists are truly similar to the query playlist, thus making the recommendations more relevant for the user.

Limitations in this work can be addressed by further analysis of title semantics, taking into consideration the individual personalities, as well as using a more suitable setup for training and fine-tuning.

REFERENCES

[1] J. Chen et al., "The Spotify Million Playlist Dataset," RecSys Challenge, 2018.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP, 2019.

[3] C. D. Manning et al., Introduction to Information Retrieval, Cambridge Univ. Press, 2008.

[4] M. Zhang and N. Hurley, "Avoiding monotony: Improving the diversity of recommendation lists," RecSys, 2008.

[5] X. Su and T. M. Khoshgoftaar, "A survey of collaborative filtering techniques," Advances in AI, 2009.

[6] H. Abdollahpouri et al., "Controlling popularity bias in recommendation," RecSys, 2017.

[7] J. Cao et al., "Web Intelligence: Research and Development," Springer, 2011.